

Tropic Air Digital Signage - Developer Manual

1. System Architecture Overview

The Tropic Air Digital Signage system is a multi-tier web application built to manage and stream media to a fleet of physical Android TVs and Tablets.

Technology Stack

- **Framework:** Next.js (App Router)
- **Database:** PostgreSQL (via Prisma ORM)
- **Storage:** Vercel Blob (for MP4s, PNGs, JPGs)
- **Authentication:** NextAuth.js (for CMS) + JWT (for TV/Tablet device authentication)
- **Hardware Clients:** Capacitor.js Android wrappers (VVSignageApp & VVTabletApp)

Core Components

1. **The CMS (Content Management System):** A web-based dashboard where Admins can upload media, build playlists, design screen-split templates, and assign content to TVs.
2. **The TV Player (/player):** A headless React application that runs inside the Android TVs. It operates as a "dumb terminal", polling the CMS for instructions, downloading the media, and caching it for offline loop playback.
3. **The Tablet Remote (/tablet):** A control panel for floor staff to instantly push emergency overrides or interactive widgets to specific TVs via Server-Sent Events (SSE).

2. Database Schema (Prisma)

The system relies on a heavily relational PostgreSQL database. Key models include:

- `\"User\"`: CMS Administrators and Agents.
- `\"TvAccount\"`: A grouped entity that physical TVs log into.
- `\"IScreen\"`: The logical representation of a physical TV.
- `\"Playlist\"` & `\"PlaylistItem\"`: Collections of Media Assets or Widgets (Weather, RSS) that play in a loop.
- `\"Template\"` & `\"ZoneMapping\"`: Allows splitting a single screen into multiple zones (e.g., Video on the left, Weather on the right).
- `\"ProofOfPlay\"`: Analytics tracking for how many seconds an ad/media item was displayed on a specific screen.

3. TV Player Lifecycle & Offline Caching

The TV Player (`src/app/player/[...id]/page.tsx`) is designed to survive network outages.

Boot Sequence

1. The TV reads `\\vsignage_cached_config\\` from its local storage and begins playing immediately (0ms delay).
2. In the background, it polls `/api/screens/[id]/config\\` to fetch the latest playlist.
3. If new media is detected, the TV quietly downloads the MP4s/Images in the background.
4. Once downloaded, it swaps the active playlist seamlessly.

Cache Eviction

The system utilizes a Service Worker (`\\sw.js`). When the active playlist changes, the Player sends an `\\EVICT_OLD_MEDIA\\` command to the Service Worker. The Service Worker scrubs the browser's Cache Storage, deleting any media that is no longer required, preventing storage bloat on the Android TVs.

4. Real-time Overrides (Server-Sent Events)

When a Tablet pushes an emergency override or a live announcement, it hits `/api/tv-accounts/[id]/override`.

To prevent the TV from waiting 60 seconds for its next polling cycle, the backend uses **Server-Sent Events (SSE)** via `/api/screens/[id]/stream`.

The SSE emitter instantly broadcasts a `\\commit\\` ping to the physical TV, forcing it to instantly reload its configuration and display the emergency message.

5. Background Jobs (Cron)

Offline Health Checks (/api/cron/check-offline)

A cron job runs on a schedule (e.g., hourly) to check the `\\lastPingAt\\` timestamp of every Screen in the database.

- If a screen has been offline for > 5 minutes, it sends an HTML Email / MS Teams Webhook alert.
- If a screen has been offline for > 30 days, it automatically scrubs it from the system to keep the database clean.

6. Android APK Configuration

The physical hardware uses standard Android web views wrapped by **Capacitor.js**.

Directory Structure

- `\\VVSignageApp\\`: The TV Application
- `\\VVTabletApp\\`: The Tablet Application

Migration & URL Management

If the backend is migrated to a new domain (e.g., moving from Vercel to a Vultr VM), the APKs **must** be recompiled.

1. Open `\\capacitor.config.json\\` in both directories.
2. Update the `\\server.url\\` parameter to point to the new domain (e.g., `\\https://signage.tropicair.com/tv\\`).
3. Rebuild the APKs in Android Studio.

Note: The backend must use HTTPS, or `\\cleartext: true\\` must be strictly defined in the Capacitor configuration for HTTP.

7. Security & API Access

- **CMS Routes:** Protected by `\\NextAuth.js\\` middleware. Administrative actions (like deleting users) require `\\role === 'ADMIN'\\`.
- **Device Routes:** Routes like `/api/tv-ping\\` and `/api/screens/[id]/config\\` bypass NextAuth. Instead, they require a cryptographically signed JWT token (`\\localStorage.getItem('vvsignage_tv_token')\\`) generated during the TV pairing process.
- **Widget Protections:** Internal widget fetchers (like the RSS proxy) actively block Server-Side Request Forgery (SSRF) by refusing to query `\\localhost\\` or internal IP addresses.